Gmail Mail Flow, Routing & Email Authentication (SPF/DKIM/DMARC) & Compliance

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Solution Overview 4

What this is . . . 4

How it works — the reference model . . . 5

Coexistence patterns (why split vs. dual delivery) . . . 5

Benefits . . . 6

Limitations & boundaries . . . 6

Deployment model . . . 6

Requirements 6

1. Verified domains with authoritative DNS access (per routed domain) . . . 7

2. Super-admin (or scoped Gmail admin) and Admin SDK API access . . . 7

3. Documented current mail flow & coexistence source of record . . . 7

4. A monitored DMARC reporting mailbox / RUA destination . . . 8

5. Inventory of legitimate senders per domain . . . 8

6. Compliance & routing policy definitions . . . 8

7. A pilot mail cohort and rollback plan for mail-flow changes . . . 9

Global Requirements — Google Workspace & Gemini Solutions 9

1. Google Workspace / Cloud Identity tenant . . . 9

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 10

3. DNS control for the routed domains . . . 10

4. Identity & provisioning source of truth . . . 10

5. A pilot scope (OU / group) and change control . . . 10

6. Security, compliance & governance stakeholders . . . 11

Implementation 11

Stage A — Assess (read-only) . . . 11

Stage B — Design . . . 12

Stage C — Configure (in the pilot OU / pilot domain first) . . . 12

Stage D — Pilot & Validate (gate before Expand) . . . 13

Stage E — Expand . . . 14

Rollback (any production change) . . . 14

Testing & Validation 14

T1 — DNS & authentication records present and correct . . . 15

T2 — SPF / DKIM / DMARC evaluation on real mail . . . 15

T3 — DMARC aggregate reports show legitimate senders aligned . . . 15

T4 — Inbound & outbound routing behaves as designed . . . 15

T5 — Content-compliance rules fire correctly . . . 16
T6 — Spam / attachment safety settings active . . . 16
T7 — Config export matches intended state (change evidence) . . . 16
T8 — Rollback rehearsal . . . 16
Troubleshooting 17
Authentication . . . 17
Routing & coexistence . . . 18
Content compliance & safety . . . 18
Scripts & API . . . 19
Best Practices 19
Authentication . . . 19
Routing & coexistence . . . 20
Compliance rules . . . 20
Operations & evidence . . . 20
Deliverables Summary 21
Documentation . . . 21
The 5 Python tools (Admin SDK / APIs; read-only or dry-run by default; no keys in code) . . . 21
Packaged output . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary

Outcome. A **designed Gmail mail plane** — inbound and outbound routing configured, split/dual delivery in place for a clean coexistence and cutover, **SPF, DKIM, and DMARC** taken all the way to **enforcement (**`p=reject`**)** with reporting, and **content-compliance and routing rules** applied consistently. The result is email that is **deliverable** (no silent spam-foldering of legitimate mail), **authenticated** (the organization's domains are hard to spoof), and **compliant** (disclaimers, archival routing, TLS enforcement, and attachment controls with an audit trail).

The problem it solves. Deliverability failures, domain spoofing, and compliance gaps almost always trace back to a mail flow that was never *designed* — MX and routing accreted over time, SPF quietly over the 10-lookup limit, DKIM unsigned, DMARC absent or stuck at `p=none`, and compliance handled ad hoc. Fixing this reactively risks breaking mail for the whole organization.

Approach — pilot-first, evidence-gated. The work follows a disciplined model: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.** Every route, host, DNS record, and rule is **named and versioned**, configured in a **pilot organizational unit first**, and validated against explicit **pass/fail gates** before it goes fleet-wide. DMARC is advanced **only on evidence** — aggregate reports must show legitimate senders aligned before the policy tightens from monitor to quarantine to reject, with a `pct` ramp limiting exposure at each step.

Why it is low-risk. The three highest-impact changes — routing/MX, DMARC advancement, and cutover — each have a **documented rollback** and are made only after DNS TTLs are lowered so a revert is fast. Five **read-only / dry-run** Python tools provide the evidence at every gate: DNS posture, DMARC alignment, config export, record validation, and a DMARC advancement planner that *never* publishes DNS on its own. No tool holds credentials or keys; automation runs least-privilege via OAuth/ADC.

Bottom line. Gmail mail flow **designed, not accidental** — routing correct, SPF/DKIM/DMARC enforced with reporting, and compliance rules in place — delivering deliverable, authenticated, and audit-ready email, at a fraction of the cost of a bespoke consulting engagement and without an outage-prone big-bang change.

Solution Overview

What this is

This solution designs and configures the **Gmail enterprise mail plane** end to end for an organization running (or moving to) Google Workspace: **inbound and outbound routing and hosts**, **split and dual delivery** for coexistence with a legacy mail system, **email authentication (SPF, DKIM, DMARC)** taken from monitor to enforcement, **content-compliance and routing rules**, and **spam and attachment controls**. The outcome is mail that is **deliverable, authenticated, and compliant** — designed on purpose rather than accreted by accident.

It is deliberately **distinct from a Workspace security baseline / DLP solution**. That work governs *data* (DLP rules, Context-Aware Access, alerting). **This** work governs the *mail plane* — how messages enter, leave, are routed, are authenticated, and are inspected by compliance rules. The two are complementary; a reference to DLP appears only where a content-compliance rule and a DLP rule could overlap and must be reconciled.

How it works — the reference model

The mail plane is designed as five cooperating layers, each configured in the Google Admin console (**admin.google.com** → **Apps** → **Google Workspace** → **Gmail**) and validated before the next is enabled:

1. **Routing & hosts.** Where mail enters and leaves. **Inbound gateway**, **outbound gateway / smart host**, **and named** Routes **and** Hosts **entries determine the delivery path. During coexistence,** split delivery **(MX at one system, some mailboxes on the other) or** dual delivery** (a copy delivered to both) keeps both systems working through the migration.

1. **Email authentication. SPF** (which hosts may send as the domain), **DKIM** (a cryptographic signature Google adds to outbound mail), and **DMARC** (the policy that ties SPF/DKIM alignment to an action and requests reports). DMARC is rolled out on the documented **enforcement journey**: `p=none` (monitor) → `p=quarantine` → `p=reject`, advancing only when aggregate reports show legitimate mail is aligned.

1. **Content compliance & routing rules.** Admin-defined rules that inspect message content, headers, and recipients to **route**, **modify**, **quarantine**, or **reject** mail — e.g. add a disclaimer, route a domain to an archive connector, quarantine attachments matching a pattern, or enforce TLS to a partner.

1. **Spam, phishing & attachment controls.** Gmail's enhanced pre-delivery message scanning, custom **approved/blocked sender** lists, **attachment** and **anti-phishing/anti-malware** safety settings, applied per organizational unit.

1. **Observability.** DMARC **aggregate (RUA) reports**, the **Admin console Email Log Search**, and the **BigQuery / Reports API** email-flow signals used to prove authentication is passing and rules are routing correctly before enforcement tightens.

Every artifact this solution creates — a route, a DMARC record, a compliance rule, a validation report, a service identity — is **named** with the convention `<Purpose> – <Scope> – <Stage>` (e.g. `DMARC – corp.example – Enforce`, `Content Compliance – Outbound Disclaimer – Pilot`, `Route – Archive Connector – Prod`) and the **same name is reused** in testing, troubleshooting, and the scripts.

Coexistence patterns (why split vs. dual delivery)

• **Split delivery** — the primary MX points at **one** system; Google is authoritative for the mailboxes hosted on Google and forwards the rest to the legacy host (or vice-versa). Used when mailboxes are being **cut over in waves** and each user lives on exactly one system at a time.

• **Dual delivery** — inbound mail is delivered to **both** systems (one authoritative, one a copy).

Used briefly during transition so users can validate Gmail while the legacy system remains the system of record. Dual delivery is **temporary** by design — it doubles storage and can duplicate auto-replies, so it is retired at cutover.

The choice, the authoritative system, and the retirement trigger are decided in **Design** and recorded so routing changes are auditable.

Benefits

• **Deliverability** — correct SPF/DKIM/DMARC alignment and clean routing stop legitimate mail from silently landing in spam or being rejected by strict receivers.

• **Anti-spoofing** — a `p=reject` DMARC policy makes the organization's domains far harder to spoof in phishing, protecting the brand and recipients.

• **Compliance** — content-compliance rules apply disclaimers, archival routing, TLS enforcement, and quarantine consistently, with an audit trail.

• **Coexistence without outages** — split/dual delivery lets mail keep flowing while mailboxes move.

• **Evidence** — DMARC reports and email-log validation prove the mail plane is healthy before and after each change.

Limitations & boundaries

• **DNS is external.** SPF/DKIM/DMARC and MX live in the authoritative **DNS** zone, which the organization (not Google) controls. This solution specifies the exact records; publishing them is a DNS change under change control.

• **Migration data movement is a separate tool.** Moving mailbox *content* from the legacy system is performed by a migration tool (**Google Workspace Migrate, GWMME, BitTitan MigrationWiz, CloudM**); this solution configures the **mail plane and coexistence routing** around that movement and defers each tool's console internals, licensing, and support to the vendor's own documentation.

• **Receiver behavior varies.** How a remote receiver treats a `quarantine`/`reject` result is the receiver's decision; DMARC states intent, it does not guarantee a specific inbox placement.

• **Gemini / preview features** referenced anywhere are flagged **"verify in your tenant"** because availability rolls out by edition and region.

Deployment model

The solution follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — **pilot-first**, with explicit **pass/fail gates** at each stage and a **documented rollback** for every production change. Mail flow, DNS, and cutover are high-impact: no DMARC policy advances, no MX change ships, and no routing rule goes fleet-wide until the pilot passes its gate.

Requirements

These are the **solution-specific** prerequisites for configuring the Gmail mail plane. They build on the shared **[Global Requirements — Google Workspace & Gemini Solutions](global-requirements-google.md)** (Workspace/Cloud Identity tenant with super-admin, source M365/Entra for coexistence, DNS control, identity source of truth, a pilot OU/group + change control, and security/compliance stakeholders). Do not re-do the shared six here — verify them, then satisfy the

additions below.

Each requirement lists **Description / Why required / How to verify / How to implement if missing.**

1. Verified domains with authoritative DNS access (per routed domain)

• **Description:** Every domain that sends or receives mail (primary, secondary, and any send-as

vanity domains) must be **verified in Google Workspace** and you must hold access to its **authoritative DNS** zone to publish MX, SPF, DKIM, and DMARC records.

• **Why required:** Routing, coexistence, and all three authentication mechanisms are DNS-published.

A domain that is unverified or whose DNS you cannot change cannot be authenticated or routed.

• **How to verify: Admin console** → **Account** → **Domains** → **Manage domains** shows each domain and its

verification state. Confirm you can create a test TXT record in each zone and see it resolve (`dig TXT _test.<domain>` / the included `mailplane_dns_audit.py`).

• **How to implement if missing:** Add and verify the domain under **Manage domains**; obtain

delegated or change-controlled access to each authoritative DNS zone before starting.

2. Super-admin (or scoped Gmail admin) and Admin SDK API access

• **Description:** A **super-admin** for the console configuration, plus a **Google Cloud project**

with the **Admin SDK API** enabled and either a delegated service account (domain-wide delegation scoped to the read-only Directory/Reports scopes used here) or an OAuth client for the read-only scripts.

• **Why required:** Routing, compliance rules, and safety settings are configured in the Admin

console; the assessment/validation scripts read directory, domain, and email-flow data through the Admin SDK / Reports API.

• **How to verify:** Confirm the admin role at **Account** → **Admin roles**; in Google Cloud confirm the

Admin SDK API is enabled and the scopes are authorized under **Security** → **API controls** → **Domain-wide delegation**.

• **How to implement if missing:** Assign the role; create/enable the Cloud project + Admin SDK API;

authorize the least-privilege scopes. **No key files are committed** — scripts use OAuth or Application Default Credentials (ADC).

3. Documented current mail flow & coexistence source of record

• **Description:** A written statement of the **current MX / mail path** for each domain, which

system is **authoritative** during coexistence, and whether **split** or **dual** delivery applies — including the legacy host's inbound smart-host FQDN and any partner/relay hosts.

• **Why required:** You cannot safely design routes and hosts without knowing the current path;

split vs. dual delivery and the authoritative system drive every routing decision and the cutover rollback.

• **How to verify:** `dig MX <domain>` for each domain (or `mailplane_dns_audit.py`) plus the legacy-side admin center; confirm the recorded path matches reality.

• **How to implement if missing:** Discover and document the current flow first; agree the authoritative system and coexistence pattern in **Design** before configuring any route.

4. A monitored DMARC reporting mailbox / RUA destination

• **Description:** A dedicated mailbox or aggregate-report processing destination to receive **DMARC RUA (aggregate) reports** — e.g. `dmarc-reports@<domain>` or a processor endpoint — and a person or process that reviews them.

• **Why required:** DMARC is advanced **only** on evidence. Aggregate reports are the evidence that legitimate senders are aligned; without a reviewed RUA destination you cannot safely leave `p=none`.

• **How to verify:** Confirm the mailbox/endpoint exists and is monitored; the DMARC record's `rua=` will point at it. The included `dmarc_rua_analyzer.py` parses the reports it receives.

• **How to implement if missing:** Create the mailbox (or provision the processor), assign an owner, and reference it in the DMARC record before publishing `p=none`.

5. Inventory of legitimate senders per domain

• **Description:** A complete list of every system that **legitimately sends as each domain** — Google Workspace, the legacy mail system (during coexistence), marketing/ESP platforms, ticketing, ERP/HR, scanners/MFPs, and relays — with each one's sending host or include mechanism.

• **Why required:** SPF has a hard **10-DNS-lookup limit** and DMARC enforcement will fail legitimate mail from any sender not aligned. The inventory is what makes an SPF record correct and DMARC safe to enforce.

• **How to verify:** Cross-check the sender list against **DMARC aggregate reports** (which reveal every source actually sending as the domain) using `dmarc_rua_analyzer.py`; reconcile gaps.

• **How to implement if missing:** Build the inventory from the ESP/app owners and from an initial `p=none` DMARC monitoring window before advancing enforcement.

6. Compliance & routing policy definitions

• **Description:** The **content-compliance and routing requirements** the mail plane must enforce — e.g. outbound disclaimers, TLS-required partner domains, archival/journaling routes, attachment restrictions, keyword/pattern quarantine — sourced from legal, security, and records stakeholders.

• **Why required:** Content-compliance rules encode policy; without the policy definitions the rules are guesses, and an over-broad rule can silently quarantine or reject legitimate mail.

• **How to verify:** Confirm each rule requirement has a named policy owner and an approved

action (route / modify / quarantine / reject) and scope (which OU/domain).

• **How to implement if missing:** Gather the definitions from the stakeholders in **Assess/Design**; reconcile any overlap with existing **DLP** rules before authoring content-compliance rules.

7. A pilot mail cohort and rollback plan for mail-flow changes

• **Description:** Beyond the shared pilot OU/group: a **pilot mail cohort** (a small OU whose users can tolerate mail-flow experimentation) and a **written rollback** for each high-impact change — MX/route change, DMARC advancement, and cutover — with the exact prior value to restore and the DNS TTL.

• **Why required:** Mail-flow, DNS, and cutover changes are the highest-impact in this solution; pilot-first validation and a known-good rollback are the safety net the house model requires.

• **How to verify:** Confirm the pilot OU exists, TTLs are lowered ahead of changes, and each planned change has a documented prior value and revert step.

• **How to implement if missing:** Create the pilot OU; lower DNS TTLs on the affected records at least one TTL-window before the change; write the rollback into the change ticket.

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized

reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model;

the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the

owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance**

stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to

enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements

before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

Implementation

Configure the Gmail mail plane following the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — **pilot-first**, with a **pass/fail gate** and a **documented rollback** at every production change. All console paths are under **admin.google.com**. Every artifact is named `<Purpose> – <Scope> – <Stage>` and that exact name is reused in testing, troubleshooting, and the scripts.

> ***High-impact warning.*** *Routing, MX/DNS, DMARC advancement, and cutover can break mail for the whole organization. Do each in the **pilot** OU first, lower DNS **TTLs** ahead of any record change, and never advance a stage until its gate passes.*

Stage A — Assess (read-only)

1. **Baseline the current mail flow.** Run the discovery script for every domain:

`python mailplane_dns_audit.py --domains-file domains.txt --out baseline` — it records current **MX, SPF, DKIM (**`google._domainkey` **and any legacy selectors), and DMARC** and flags the SPF 10-lookup limit. Save the output as the `Mailplane Baseline – All Domains – Assess` artifact.

1. **Enumerate routing surface.** Run `mailplane_config_export.py --section routing --out routing`

(read-only Admin SDK) to export existing **Hosts, Routes, and default routing** so nothing is overwritten blindly.

1. **Start DMARC monitoring if not already present.** If any domain lacks DMARC, publish a

monitor-only record now so evidence starts accruing while you design (see Stage C, step C1 — begin at `p=none`). Name it `DMARC – <domain> – Monitor`.

1. **Gate A (Assess → Design):** baseline captured for **every** domain; RUA reports arriving at the

monitored mailbox; legitimate-sender inventory started. **Fail** → resolve DNS/monitoring access before proceeding.

Stage B — Design

1. **Choose the coexistence pattern per domain** — **split** or **dual** delivery — the

authoritative system, and the **retirement trigger** for dual delivery. Record as `Coexistence Design – <domain> – Design`.

1. **Design routing & hosts.** Define each **Host** (legacy inbound smart host, archive/journal

connector, partner relay) and each **Route** with its name, match, and action. Reserve names now: e.g. `Host – Legacy Smart Host – Prod`, `Route – Archive Connector – Prod`, `Route – Split Delivery to Legacy – Prod`.

1. **Design authentication targets.** For each domain define the intended **SPF** record (all

legitimate senders within the 10-lookup limit), confirm **DKIM** will use the Google 2048-bit key with selector `google`, and set the **DMARC end-state** (`p=reject`, `rua=`, optional `ruf=`, `pct=` ramp). Record as `SPF – <domain> – Target`, `DKIM – <domain> – Target`, `DMARC – <domain> – Enforce`.

1. **Design compliance/routing rules** from the policy definitions: name, scope (OU/domain), match,

and action (route/modify/quarantine/reject) — e.g. `Content Compliance – Outbound Disclaimer – Pilot`, `Content Compliance – TLS Required Partner – Pilot`, `Content Compliance – Attachment Quarantine – Pilot`. Reconcile any overlap with existing DLP rules.

1. **Gate B (Design → Configure):** every route/host/record/rule is named and approved; rollbacks and

TTL reductions are written into the change ticket.

Stage C — Configure (in the pilot OU / pilot domain first)

C1 — Email authentication

1. **DKIM. Apps** → **Google Workspace** → **Gmail** → **Authenticate email** → select the domain →

Generate new record (2048-bit, selector `google`) → publish the provided `google._domainkey.<domain>` TXT (CNAME/TXT per your DNS) → return and **Start authentication**. Confirm with `mailauth_validate.py --domain <domain> --check dkim`. Artifact: `DKIM – <domain> – Prod`.

1. **SPF.** Publish the designed SPF TXT for the domain (typically `v=spf1 include:_spf.google.com

~all plus each legitimate sender's include, kept ≤**10 lookups**). Verify with mailauth_validate.py --domain <domain> --check spf. Artifact: SPF – <domain> – Prod`.

1. **DMARC — monitor.** Publish `_dmarc.<domain>` TXT `v=DMARC1; p=none; rua=mailto:<rua>; adkim=r;

aspf=r (start relaxed). Artifact: DMARC – <domain> – Monitor. **Do not advance yet** – collect at least one to two weeks of aggregate reports and reconcile senders with dmarc_rua_analyzer.py`.

C2 — Routing & hosts (coexistence)

1. **Add hosts. Gmail** → **Hosts** → **Add route** → create each designed **Host** (legacy smart host, archive connector, partner relay). Name each exactly as designed (`Host – Legacy Smart Host – Prod`).

1. **Configure inbound gateway** (if a gateway/relay fronts Google): **Gmail → Spam, phishing, and malware → Inbound gateway** → add the gateway IPs so Gmail trusts and correctly evaluates SPF for relayed mail.

1. **Configure split or dual delivery. Gmail** → **Routing** → **Routing** → add the rule for the chosen pattern:

• **Split:** route mail for legacy-hosted mailboxes to `Host – Legacy Smart Host – Prod` (`Route – Split Delivery to Legacy – Prod`), Google authoritative for its own mailboxes.

• **Dual:** deliver to Google **and** add a copy/recipient to the legacy host (`Route – Dual Delivery Copy – Prod`) — mark it **temporary**, tie it to the retirement trigger.

1. **Outbound gateway / smart host** (if outbound must egress via a relay): **Gmail → Routing → Outbound gateway** → set the smart host. Artifact: `Host – Outbound Smart Host – Prod`.

C3 — Content compliance & safety

1. **Content-compliance & routing rules. Gmail** → **Compliance** → **Content compliance** (and **Routing** / **Objectionable content** as needed) → create each designed rule scoped to the **pilot OU** first, using the exact names (`Content Compliance – Outbound Disclaimer – Pilot`, `Content Compliance – TLS Required Partner – Pilot`, `Content Compliance – Attachment Quarantine – Pilot`). For TLS-required, set **Require secure transport (TLS)** on the route.

1. **Spam, phishing & attachments. Gmail** → **Spam, phishing, and malware** → enable enhanced pre-delivery scanning and the attachment/anti-phishing/anti-malware safety toggles for the pilot OU; add any **approved/blocked senders** lists. Artifact: `Safety – Pilot OU – Pilot`.

1. **Export the configured state** for the change record: `mailplane_config_export.py --section all --out configured` — the export is the reviewable artifact and the rollback reference.

Stage D — Pilot & Validate (gate before Expand)

1. Send the **validation test matrix** (see `05-testing.md`) from and to the pilot OU: internal, inbound external, outbound external, a disclaimer-triggering message, a TLS-partner message, and a quarantine-triggering attachment.

1. Confirm authentication with `mailauth_validate.py --domain <domain> --check all` and inspect

delivery with **Email Log Search**.

1. **Advance DMARC to quarantine — only if** `dmarc_rua_analyzer.py` shows **≥ ~98% of legitimate

volume aligned** and every source is accounted for: update `_dmarc.<domain>` to `p=quarantine; pct=100` (optionally ramp `pct=25→50→100`). Artifact: `DMARC – <domain> – Quarantine`.

1. **Gate D (pilot pass/fail):**

• **PASS:** all six test flows behave as designed; authentication passes; rules route/modify/

quarantine correctly; no legitimate mail lost; DMARC aggregate alignment ≥ target.

• **FAIL / ROLLBACK:** revert the offending route/rule to its exported prior state; roll DMARC

back to the previous p= value (DNS TTL was pre-lowered); do **not** expand.

Stage E — Expand

1. Widen each validated route, rule, and safety setting from the pilot OU to the **remaining OUs /

fleet**, renaming the stage token `Pilot → Prod` (e.g. `Content Compliance – Outbound Disclaimer – Prod`).

1. **Advance DMARC to reject** once fleet-wide alignment holds at the quarantine target with no

collateral: `_dmarc.<domain>` → `p=reject; pct=100`. Artifact: `DMARC – <domain> – Enforce`.

1. **Retire dual delivery** at the recorded trigger (cutover complete): remove

`Route – Dual Delivery Copy – Prod` and update MX to the final authoritative path. Re-run `mailplane_dns_audit.py` to confirm the final state.

1. **Restore DNS TTLs** to normal values and archive the final `mailplane_config_export.py` output as

the steady-state baseline.

Rollback (any production change)

• **DNS/DMARC/MX:** re-publish the exact prior record captured in the Stage A/C exports; because TTLs

were lowered pre-change, propagation is fast. DMARC steps down one policy level at a time (`reject → quarantine → none`).

• **Routing/compliance rule:** disable or restore the rule to its exported prior state

(`configured`/`baseline` export). Rules are scoped to OUs, so a rollback is limited to that OU.

• **Cutover:** if a wave's cutover fails, re-point the affected mailboxes' delivery back to the

authoritative legacy host per the recorded split/dual design and re-validate before retrying.

Testing & Validation

Run these in the **pilot OU** before Gate D and again after **Expand**. Each procedure states the **expected pass** and the **fail signal**. Use the exact artifact names from `03-implementation.md`. The included

Python tools are read-only/report-only and are safe to run repeatedly.

T1 — DNS & authentication records present and correct

- **How:** `python mailplane_dns_audit.py --domains-file domains.txt --out validate`
- **Pass:** every domain shows the intended **MX**, an **SPF** record within the **10-lookup** limit,

a resolving **DKIM** `google._domainkey` record, and a **DMARC** record at the expected policy level for the current stage.

- **Fail:** missing/duplicate SPF, SPF > 10 lookups, DKIM not resolving, or DMARC policy not at the

stage's expected value. Fix in DNS / Admin console before advancing.

T2 — SPF / DKIM / DMARC evaluation on real mail

- **How:** `python mailauth_validate.py --domain <domain> --check all`, then send a test message from

the domain to an external address you control and inspect **Authentication-Results** / original-message headers.

- **Pass:** headers show `spf=pass`, `dkim=pass`, and `dmarc=pass` with **alignment** (the DKIM `d=`

and/or SPF domain aligns with the From domain).

- **Fail:** any of `spf/dkim/dmarc=fail` or `dmarc=... (p=none)` misalignment. Reconcile the sending

host against the SPF include list and DKIM signing before advancing DMARC.

T3 — DMARC aggregate reports show legitimate senders aligned

- **How:** collect RUA reports at the monitored mailbox for the monitoring window, then

`python dmarc_rua_analyzer.py --input reports/ --out rua-summary`

- **Pass:** ≥ **~98%** of legitimate volume passes DMARC (aligned), and **every** sending source in

the report is a known, approved sender from the inventory.

- **Fail:** legitimate sources failing alignment, or an **unknown** source sending volume. Do **not**

advance to `quarantine`/`reject` until resolved (add the sender to SPF/DKIM or investigate spoofing).

T4 — Inbound & outbound routing behaves as designed

- **How:** send the six-flow matrix — internal, inbound external, outbound external, disclaimer

trigger, TLS-partner, quarantine-attachment — and trace each in **Gmail** → **Email Log Search**.

- **Pass:** each message follows the **named route** (`Route – …`), reaches the correct system under

the chosen split/dual pattern, and no message loops or double-delivers unexpectedly.

• **Fail:** mail delivered to the wrong system, dropped, or duplicated (beyond intended dual delivery). Correct the route ordering/match and re-test.

T5 — Content-compliance rules fire correctly

• **How:** send messages that should and should **not** trigger each rule (`Content Compliance – Outbound Disclaimer – Pilot`, … `– TLS Required Partner – Pilot`, … `– Attachment Quarantine – Pilot`).

• **Pass:** triggering messages are modified/routed/quarantined as designed; non-triggering messages pass untouched; the TLS-partner message is delivered only over TLS.

• **Fail:** false positives (legitimate mail quarantined/rejected) or false negatives (policy mail not caught). Tighten/loosen the match; verify rule order; reconcile with DLP overlap.

T6 — Spam / attachment safety settings active

• **How:** send an EICAR-style test attachment and a known-benign attachment to a pilot mailbox; confirm approved/blocked sender behavior.

• **Pass:** malicious/blocked content is caught per the `Safety – Pilot OU – Pilot` settings; benign mail from approved senders is delivered.

• **Fail:** malicious content delivered or benign approved-sender mail blocked. Adjust the safety toggles and lists for the OU.

T7 — Config export matches intended state (change evidence)

• **How:** `python mailplane_config_export.py --section all --out validate` and diff against the approved design.

• **Pass:** exported routes, hosts, and rules match the named, approved artifacts exactly.

• **Fail:** drift (an unnamed or unexpected route/rule). Investigate the change and reconcile before declaring the gate passed.

T8 — Rollback rehearsal

• **How:** in the pilot OU, disable a test rule / step DMARC down one level, confirm reversion, then restore.

• **Pass:** the prior state is restored quickly (TTLs were pre-lowered) and mail flow returns to baseline. This proves the rollback documented in the change ticket actually works.

• **Fail:** reversion does not restore prior behavior — fix the rollback procedure **before** any fleet-wide change.

Gate D summary: T1–T6 pass in the pilot OU and T3 shows alignment ≥ target **before** advancing DMARC or expanding fleet-wide. After Expand, re-run T1–T3 and T7 to confirm the steady state.

Troubleshooting

Common mail-plane failure modes and their resolutions. Use the **named artifacts** from `03-implementation.md` and the read-only scripts to diagnose. **Email Log Search** (**Gmail** → **Email Log Search**) is the primary tracing tool.

Authentication

Legitimate mail lands in spam after publishing DMARC

- **Likely cause:** A legitimate sender is **not aligned** (not in SPF or not DKIM-signing as the domain) and DMARC is already at `quarantine`/`reject`.
- **Resolve:** Run `dmarc_rua_analyzer.py` to find the failing source, add it to the SPF include list or enable DKIM for it, **or** step DMARC back to `p=none` (rollback) until it aligns. Never advance DMARC before T3 shows alignment ≥ target.

SPF returns `permerror` / "too many DNS lookups"

- **Likely cause:** SPF exceeds the **10-lookup** limit (too many `include:`/a/mx/`exists:`). `mailplane_dns_audit.py` / `mailauth_validate.py` reports the count.
- **Resolve:** Remove decommissioned senders, consolidate includes, and only as a last resort use a maintained SPF-flattening service. Re-validate with T1/T2.

SPF permerror from multiple SPF records

- **Likely cause:** More than one `v=spf1` TXT record on the domain (only one is permitted).
- **Resolve:** Merge into a single record. `mailauth_validate.py --check spf` flags this.

DKIM shows `fail` or does not resolve

- **Likely cause:** DKIM was generated in the console but **authentication wasn't started**, the `google._domainkey` record isn't published/propagated, or a wrong selector.
- **Resolve:** In **Authenticate email**, confirm the record is published (matching selector `google`), then **Start authentication**. Verify with `mailauth_validate.py --check dkim`. Allow for DNS propagation (TTL).

DMARC record present but no aggregate reports arriving

- **Likely cause:** Missing/incorrect `rua=` mailto, or the reporting mailbox isn't monitored, or (for a third party) their domain isn't authorized to send you reports.
- **Resolve:** Confirm `rua=mailto:` points at the monitored mailbox; for external processors add the

cross-domain authorization TXT the processor documents (**verify per your processor's docs**).

Routing & coexistence

Mail delivered to the wrong system (split delivery)

• **Likely cause:** Route match/order sends mail for a migrated user to the legacy host (or vice versa), or the authoritative system was misidentified.

• **Resolve:** Check `Route – Split Delivery to Legacy – Prod` match and **ordering** in **Gmail** → **Routing**; put the most specific route first; confirm each user's authoritative system.

Duplicate messages / duplicate auto-replies

• **Likely cause: Dual delivery** left on beyond its retirement trigger, or overlapping routes both delivering.

• **Resolve:** Retire `Route – Dual Delivery Copy – Prod` at the recorded trigger; check for overlapping routes with `mailplane_config_export.py` (T7).

Mail loops between Google and the legacy host

• **Likely cause:** Both systems forward to each other (each thinks the other is authoritative).

• **Resolve:** Define a single authoritative system per mailbox; ensure the non-authoritative side forwards, it does not re-route back. Trace with Email Log Search.

Relayed inbound mail fails SPF/DMARC

• **Likely cause:** A gateway/relay fronts Google but isn't registered, so Gmail evaluates SPF against the relay, not the true origin.

• **Resolve:** Add the gateway IPs under **Spam, phishing, and malware** → **Inbound gateway**.

Content compliance & safety

Legitimate mail quarantined/rejected by a content rule (false positive)

• **Likely cause:** An over-broad match in a `Content Compliance – … – Pilot/Prod` rule, or a rule ordered ahead of an exception.

• **Resolve:** Narrow the match, add an exception, or re-order; prefer **quarantine** over **reject** so it's reversible. Reconcile with any overlapping **DLP** rule.

Outbound disclaimer duplicated or missing

• **Likely cause:** Rule scope/order wrong, or the append/prepend condition matches replies too.

• **Resolve:** Scope `Content Compliance – Outbound Disclaimer – Prod` to outbound only and exclude internal/reply threads as designed; re-test (T5).

TLS-required mail to a partner bounces

• **Likely cause:** The partner's receiving server no longer offers/accepts the required TLS, so the `TLS Required Partner` route hard-fails.

• **Resolve:** Confirm the partner's TLS status; temporarily relax to opportunistic TLS only under change control if policy allows, and notify the partner.

Scripts & API

Admin SDK export fails with auth / permission errors

• **Likely cause:** Missing ADC/OAuth, scopes not authorized in **domain-wide delegation**, or the account lacks the read-only role.

• **Resolve:** Authorize the read-only Directory scope, set `GOOGLE_APPLICATION_CREDENTIALS` or pass `--oauth-client`, or run `mailplane_config_export.py --offline` to validate the manifest only. Never commit key files.

DNS scripts return nothing / stale results

• **Likely cause:** `dnspython` not installed (nslookup fallback in use) or DNS caching within TTL.

• **Resolve:** `pip install dnspython`; account for TTL after any record change (lower TTLs before changes so re-checks reflect reality quickly).

`dmarc_rua_analyzer.py` *finds no reports*

• **Likely cause:** `--input` points at an empty/wrong path, or reports haven't accrued yet.

• **Resolve:** Point `--input` at the directory receiving RUA reports; allow a full monitoring window (days) at `p=none` before expecting representative volume.

Best Practices

Durable practices for running the Gmail mail plane — how to keep routing, authentication, and compliance correct over time, and how to move DMARC to enforcement **without breaking legitimate senders**.

Authentication

• **Always monitor before you enforce.** Live at `p=none` long enough (typically 1–4 weeks, longer for large sender fleets) to see a full billing/marketing cycle in aggregate reports before moving to `quarantine`, and again before `reject`. Evidence, not the calendar, gates each step.

• **Ramp with `pct`.** When advancing to `quarantine`/`reject`, ramp `pct=25` → `50` → `100` so a misconfiguration affects only a fraction of mail while you watch the reports.

• **Keep SPF under the 10-lookup limit.** Every `include:` and nested lookup counts. Prefer a small

set of includes; remove decommissioned senders; use SPF-flattening only as a documented last resort (it must be maintained). `mailplane_dns_audit.py` reports the lookup count.

- **Sign with 2048-bit DKIM and one clear selector (`google`).** Don't leave stale legacy selectors published once the legacy system stops sending — they widen the spoofing surface.
- **Start DMARC alignment relaxed (`adkim=r; aspf=r`), tighten only if needed.** Strict alignment breaks common legitimate patterns (subdomains, some ESPs); most organizations reach `p=reject` safely on relaxed alignment.
- **Publish DMARC on subdomains too.** Add a subdomain policy (`sp=`) or explicit `_dmarc.<sub>` records so an unused subdomain can't be spoofed under a lenient parent.

Routing & coexistence

- **Name and version every route, host, and rule** with `<Purpose> – <Scope> – <Stage>` and keep the exported config as the change record. An unnamed route is a future outage no one can explain.
- **Order matters.** Routing and content-compliance rules evaluate in order; put the most specific rules first and confirm ordering with a test message, not assumption.
- **Treat dual delivery as temporary.** Tie it to an explicit retirement trigger; leaving it on doubles storage and duplicates auto-replies and DMARC accounting.
- **Lower DNS TTLs before any MX/SPF/DMARC change** and restore them after — this makes rollback fast and is the difference between a 5-minute revert and an hour of bad mail.
- **Trust relayed mail correctly.** If a gateway fronts Google, register it as an **inbound gateway** so SPF/DMARC is evaluated against the true origin, not the relay.

Compliance rules

- **Prefer route/modify/quarantine over reject** for content rules where a false positive would lose legitimate mail — quarantine is reversible; reject is not.
- **Reconcile with DLP.** A content-compliance rule and a DLP rule can both match the same message. Decide precedence deliberately so mail isn't double-handled or silently dropped.
- **Scope by OU and pilot first.** Never author a fleet-wide content rule directly; prove it in the pilot OU, then promote `Pilot → Prod`.
- **Enforce TLS to partners with a documented list**, and monitor for delivery failures when a partner's TLS lapses.

Operations & evidence

• **Review DMARC aggregate reports on a schedule**, not just during rollout — a new unaligned source usually means a new legitimate app was deployed (add it) or an active spoofing attempt (investigate).

• **Keep the legitimate-sender inventory current.** It is the single source of truth that keeps SPF correct and DMARC safe; update it whenever an app that sends mail is added or retired.

• **Re-baseline after every change** with `mailplane_dns_audit.py` + `mailplane_config_export.py` and archive the output — the exports are your rollback reference and audit evidence.

• **Run all scripts read-only/dry-run by default**; require an explicit flag for any change, use OAuth/ADC (never committed keys), and keep the RUA/report data access least-privilege.

Deliverables Summary

Everything included in this solution, on one page.

Documentation

• **Solution Overview** — the Gmail mail plane: routing/hosts, split & dual delivery, SPF/DKIM/DMARC, content-compliance/routing rules, and the enforcement journey.

• **Requirements Guide** — the shared Google global requirements plus seven solution-specific prerequisites (verified domains + DNS, super-admin + Admin SDK access, documented current flow, RUA mailbox, legitimate-sender inventory, compliance policy, pilot cohort + rollback).

• **Implementation Guide** — Assess → Design → Configure → Pilot → Validate → Expand, pilot-first, every route/host/record/rule named `<Purpose> – <Scope> – <Stage>`, with pass/fail gates and a documented rollback for every high-impact change.

• **Testing & Validation** — eight procedures (T1–T8) each with an explicit pass and fail signal, culminating in Gate D.

• **Troubleshooting** — the common mail-plane failure modes and their resolutions.

• **Best Practices** — durable practices, especially moving DMARC to enforcement without breaking legitimate senders.

• **Executive Summary** — the outcome, approach, and why it is low-risk.

The 5 Python tools (Admin SDK / APIs; read-only or dry-run by default; no keys in code)

1. `mailplane_dns_audit.py` — *Assessment / discovery (read-only).* Audits **MX, SPF, DKIM, and DMARC** DNS posture across all domains, counts SPF DNS lookups against the 10-lookup limit, and flags gaps. Produces the `Mailplane Baseline` artifact.

1. `dmarc_rua_analyzer.py` — *Validation / report generator (read-only).* Parses **DMARC aggregate (RUA) reports (`.xml/.gz/.zip`), aggregates volume by sending source, and computes the** % of legitimate volume aligned** — the evidence gate for advancing DMARC.

1. `mailplane_config_export.py` — *Configuration / export helper (read-only).* Exports readable mail-plane config (domains/aliases) via the **Admin SDK** and validates that supplied routing/ compliance artifact **names follow the convention** — the change-record and rollback reference.

1. `mailauth_validate.py` — *Policy / health check (read-only).* Validates a domain's **SPF/DKIM/DMARC** records for correctness and against the intended enforcement stage; produces a per-check PASS/FAIL report.

1. `dmarc_policy_planner.py` — *Scope / config helper (dry-run by default).* Computes the **next safe DMARC record (`none` → `quarantine` → `reject`) given measured alignment and a `pct` ramp, and returns a** SAFE/HOLD** decision. It only prints the record — it never publishes DNS.

Packaged output

- `GWMAIL-Solution.pdf` — the compiled solution document (all sections above).
- `GWMAIL-Scripts.zip` — the five Python tools.

All scripts are Python 3.10+, argparse-driven with `--help`, standard-library-first (optional `dnspython`, `google-api-python-client`/`google-auth` where noted), read-only or `--dry-run` by default, and contain **no hardcoded credentials or keys** (OAuth / ADC only).

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

dmarc_policy_planner.py

dmarc_rua_analyzer.py

mailauth_validate.py

mailplane_config_export.py

mailplane_dns_audit.py